

subqueries

what is a subquery?

a query within a query

query1(query2)

in the above query1 is the outer query

query2 is the inner query.

-- customers table and an orders table

-- Find all the customers who have not placed even a single order.

you need customer_ids from customers table

which are not present in orders table

customers customer_id

orders order_customer_id

-- subquery

select * from customers where customer_id not in
(select order_customer_id from orders);

customers orders

left join

check for nulls on the right

-- left outer join

select c.*
from customers c
left join orders o
on c.customer_id = o.order_customer_id
where o.order_customer_id is null;

-- except

select customer_id from customers
except
select order_customer_id from orders;

-- not exists (correlated subquery)

select * from customers c where not exists
(select * from orders o where o.order_customer_id = c.customer_id);

customers table has 100 records

1
2
3
4
.
.
100

exists not exists

=====

-- all the customers who have placed atleast one order

-- subquery

select * from customers where customer_id in
(select order_customer_id from orders);

-- using inner join

select distinct(customer_id)
from customers c
inner join orders o
on c.customer_id = o.order_customer_id

-- using intersect

select customer_id from customers
intersect
select order_customer_id from orders;

-- correlated subquery

select * from customers c
where exists (select * from orders o
 where o.order_customer_id = c.customer_id);

-- we need to find avg number of order items in each order.

-- and we need to find avg order amount.

order_id = 2

3 items

laptop

mobile

charger

order_id, num_of_items, order_amount

1,	1,	300
2,	3,	580
.		
.		
.		

```
select order_item_order_id as order_id,  
count(*) as num_order_items,  
sum(order_item_subtotal) as order_amount  
from order_items  
group by order_item_order_id;
```

```
select avg(num_order_items) as avg_order_items,  
avg(order_amount) as avg_order_amount  
from (select order_item_order_id as order_id,  
count(*) as num_order_items,  
sum(order_item_subtotal) as order_amount  
from order_items  
group by order_item_order_id) x;
```

-- find avg number of orders placed by each customer

customer_id, number_of_orders_placed

1, 4
2, 3
3, 7
4, 1

```
select order_customer_id, count(*) as total_orders  
from orders  
group by order_customer_id;
```

```
select avg(total_orders)  
from (select order_customer_id, count(*) as total_orders  
from orders  
group by order_customer_id) x;
```

I want to find big orders

all the orders where order items are > avg_order_items
and order amount > avg_order_amount

1, 3, 500
2, 5, 800

A (500)
order_id,num_order,items,order_amount
1,1,299
2,3,580
4,4,700
...

B (ONLY 1 ROW)

avg_order_items,	avg_order_amount
2.9983 ,	597.6322996016944

order_id,num_order,items,order_amount,avg_order_items,avg_order_amount
1,1,299,2.9983,597.6322996016944
2,3,580,2.9983,597.6322996016944

```
select * from
(select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal) as order_amount
from order_items
group by order_item_order_id) a
join
(select avg(num_order_items) as avg_order_items,
avg(order_amount) as avg_order_amount
from (select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal) as order_amount
from order_items
group by order_item_order_id) x) b
on num_order_items > avg_order_items
and order_amount > avg_order_amount;
```

```
select *
from a
join b
on
```

```
-- from
-- join
-- where
-- having
```

```
select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal) as order_amount
from order_items
group by order_item_order_id
having count(*) > (select avg(num_order_items) as avg_order_items
from (select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal)
from order_items
group by order_item_order_id) x)
and order_amount > (select avg(order_amount) as avg_order_amount
from (select order_item_order_id as order_id,
```

```
count(*) as num_order_items,  
sum(order_item_subtotal) as order_amount  
from order_items  
group by order_item_order_id) x)
```

```
select order_item_order_id as order_id,  
count(*) as num_order_items,  
sum(order_item_subtotal) as order_amount  
from order_items  
group by order_item_order_id  
having count(*) > 2.9983  
and order_amount > 597.6322996016944
```

=====

```
select avg(total_orders)  
from (select order_customer_id, count(*) as total_orders  
from orders  
group by order_customer_id) x;
```

-- we want to find premium customers

```
number of orders > avg number of orders  
select order_customer_id, count(*)  
from orders  
group by order_customer_id  
having count(*) > (select avg(total_orders)  
from (select order_customer_id, count(*) as total_orders  
from orders  
group by order_customer_id) x);
```

-- as part of select in orders table I want one extra column which indicates
-- the avg number of orders each customer places.

```
select *, (select avg(total_orders)  
from (select order_customer_id, count(*) as total_orders  
from orders  
group by order_customer_id) x) as avg_orders  
from orders;
```

```
-- from  
-- join  
-- where  
-- having  
-- select
```

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary DECIMAL(10, 2)  
);
```

```
INSERT INTO employees (emp_id, name, salary) VALUES  
(1, 'Amit', 60000.00),  
(2, 'Sneha', 75000.00),  
(3, 'Raj', 50000.00),  
(4, 'Priya', 80000.00),  
(5, 'Vijay', 45000.00);
```

-- find employees with above avg salaries

```
select * from employees
```

```
where salary > (select avg(salary) as avg_salary from employees)
```

There are 3 main types of subqueries

1. scalar subquery: a subquery that returns just one row and one column. in general terms a subquery which returns just one value.

where / having

>

<

=

2. Multiple rows / multiple columns

from

join

3. correlated subquery: a subquery that depends on the results of the outer query.

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary DECIMAL(10, 2),  
    department_id INT  
);
```

```
INSERT INTO employees (emp_id, name, salary, department_id) VALUES
(1, 'Amit', 60000.00, 1),
(2, 'Sneha', 75000.00, 2),
(3, 'Raj', 50000.00, 1),
(4, 'Priya', 80000.00, 2),
(5, 'Vijay', 45000.00, 1),
(6, 'Anita', 70000.00, 3),
(7, 'Sunil', 65000.00, 3),
(8, 'Pooja', 55000.00, 2);
```

-- find employees earning more than avg salary in their department

```
dept_id,avg_salary
1,51666
2,70000
3,67500
```

```
e
(1, 'Amit', 60000.00, 1),52000
(2, 'Sneha', 75000.00, 2),70000
(3, 'Raj', 50000.00, 1),52000
(4, 'Priya', 80000.00, 2),70000
(5, 'Vijay', 45000.00, 1),52000
(6, 'Anita', 70000.00, 3),80000
(7, 'Sunil', 65000.00, 3),80000
(8, 'Pooja', 55000.00, 2),70000
```

```
d
dept_id,avg_salary
1,51666
2,70000
3,67500
```

```
select emp_id, name, salary, e.department_id, avg_salary
from employees e
inner join
(select department_id,avg(salary) as avg_salary
from employees group by department_id) d
on e.department_id = d.department_id;
```

```
select emp_id, name, salary, e.department_id, avg_salary
from employees e
inner join
(select department_id, avg(salary) as avg_salary
from employees
group by department_id) d
on e.department_id = d.department_id
where salary > avg_salary;
```

-- correlated subquery

```
SELECT
    *
FROM
    employees e
WHERE
    salary > (SELECT
                AVG(salary)
            FROM
                employees
            WHERE
                department_id = e.department_id);
```